Lecture 03.03: Odd or Even (Parity Checking Logic)

📌 Executive Summary

Algorithmic parity analysis comparing standard modulo arithmetic against low-level bitwise masking.

🔬 Theoretical Foundations & Principles

1. Parity Invariants

An integer $x \in \mathbb{Z}$ is even if divisible by 2 without remainder; otherwise, it is odd.

$$\text{parity}(x) = x \pmod 2 = \begin{cases} 0 & \iff x \text{ is even} \\ 1 & \iff x \text{ is odd} \end{cases}$$

2. Bitwise Parity Optimization

In binary 2's complement representation, the least significant bit (LSB) indicates parity:

$$x \,\&\, 1 = \begin{cases} 0 & \iff x \text{ is even} \\ 1 & \iff x \text{ is odd} \end{cases}$$

Bitwise testing operates directly on CPU register bits without executing division circuitry.

💻 Production Implementation & Code

```
def check_parity_modulo(n: int) -> str:
    return "Even" if n % 2 == 0 else "Odd"

def check_parity_bitwise(n: int) -> str:
    return "Even" if (n & 1) == 0 else "Odd"
```

```
for val in [0, 14, 27, -8, -13]:
    print(f"Number: {val:4d} | Modulo: {check_parity_modulo(val):4s} | Bitwise: {c
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.